SIR PADAMPAT SINGHANIA UNIVERSITY

WEB DEVELOPMENT / MINI PROJECT PROPOSAL

E-Library

A Smart Digital Library and Online Reading Platform

Student name	Padam Kishore
Enrollment number	25CS003692
Programme	B.Tech CSE (AI-ML)
Semester	III
University	Sir Padampat Singhania University
Course	Web Development
Submission date	5 November 2026

Project overview

E-Library brings searchable book discovery, integrated PDF/EPUB reading and administrator-managed content into one responsive web application. The proposal builds on an existing implementation and introduces a small catalogue/report module to demonstrate the requested course technologies without restructuring the main library.

Repository: github.com/padam421/E-LIBRARY

Live library: e-library-c9t.pages.dev

Primary stack: HTML5, CSS3, JavaScript, Node.js/Express and MySQL.
Supplementary module: React, Bootstrap and PHP catalogue reporting.

03 / WEB DEVELOPMENT

Problem statement

Students and readers often rely on scattered PDF/EPUB files, shared links and separate applications to locate and read learning materials. This makes it difficult to search resources, resume reading and maintain a consistent digital collection. Library administrators also need a reliable way to organize book metadata, control visibility and manage access. E-Library addresses these problems through a centralized, responsive web application that combines searchable book discovery, browser-based reading, secure sign-in and an administrator dashboard backed by a MySQL database.

Intended users

Students and general readers use the public catalogue and reading interfaces. Authorized administrators maintain the collection, metadata and visibility settings.

04 Objectives of the project

1. Develop a responsive digital library for desktop and mobile browsers.

2. Enable book discovery by title, author, category and PDF/EPUB format.

3. Provide browser-based PDF and EPUB reading with navigation and saved progress.

4. Implement Google sign-in, authenticated sessions and role-based administrator access.

5. Support MySQL-backed book creation, viewing, updating, deletion and bulk import.

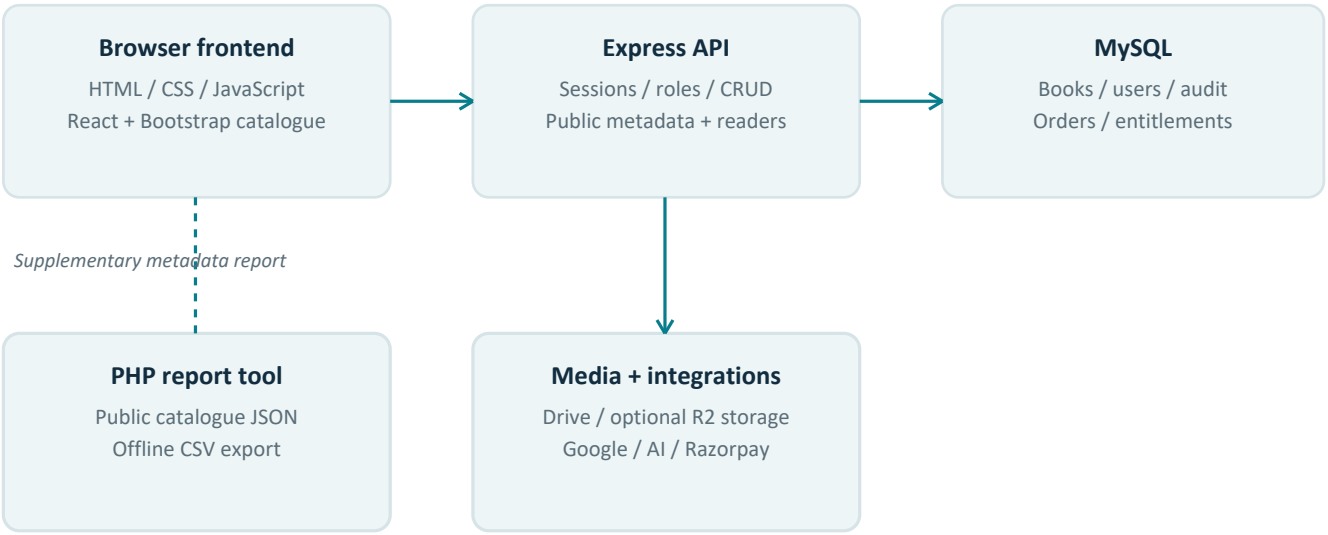
6. Apply the course technologies through a small React/Bootstrap catalogue and a supplementary PHP CSV report tool.

7. Test major workflows and maintain source code, SQL schema and setup instructions on GitHub.

05 / WEB DEVELOPMENT

Proposed solution

Provide a centralized browser-based platform with a public discovery layer, authenticated reading and a protected administrator workspace. The frontend exchanges JSON with an Express API; MySQL stores structured records while book assets remain in configured media storage.



Role	Main workflow
Visitor	Browse public titles, inspect metadata and open permitted previews.
Signed-in reader	Read permitted full documents and resume the reading experience.
Administrator / owner	Create, update, import and delete book records; manage visibility and authorized administration.

Small, practical extension

A separate React/Bootstrap catalogue reuses the existing public API. A supplementary PHP CLI tool exports public metadata to CSV. The main library interface, reader routes, database schema and primary Express backend are retained.

06 / WEB DEVELOPMENT

Key features and functionalities

Module	Functionality
Discovery	Search books, view covers and metadata, and browse curated categories.
React catalogue	Filter by title/author/category/format, view counts, paginate and print the current page.
Integrated reading	PDF and EPUB viewers, page/chapter navigation and saved reading progress.
Access management	Public previews, Google sign-in, signed sessions and administrator role checks.
Administration	Book CRUD, bulk import, public/private visibility and activity logs.
PHP reports	Export catalogue metadata to CSV with Unicode/quoting support and formula escaping.
Existing integrations	AI reading assistance, configurable premium access and Razorpay support/payments.
Usability / discovery	Responsive layouts, theme controls and SEO-ready public book pages.

07 Proposed technologies

Technology	Role in this project
HTML5	Semantic structure, forms, tables, book pages and reading interfaces.
CSS3	Responsive layouts, Flexbox/Grid, media queries, themes and print styles.
JavaScript / AJAX / JSON	DOM events, validation, Fetch requests and JSON client-server exchange.
React 18.3.1	Catalogue state, search/category/format filters, statistics and pagination.
Bootstrap 5.3.8	Catalogue grid, responsive table, forms, buttons and layout utilities.
Node.js / Express	Primary backend API, routes, middleware, authentication and business logic.
PHP 8.1+	Supplementary command-line conversion of public catalogue JSON to safe CSV.
MySQL	Book/user metadata, administrator audit records, payment records and CRUD.
Git / GitHub	Version control, repository sharing and automated syntax/PHP checks.

08 / WEB DEVELOPMENT

Expected outcomes

Deliver a functional digital library that reduces fragmented resource discovery, supports in-browser reading and gives administrators a consistent content-management workflow. The result should be demonstrable from desktop and mobile browsers with a documented setup and reproducible source code.

Acceptance criteria

- Readers can locate a title, open its details and use the available PDF/EPUB reading path.

- Authorized administrators can maintain book records through the existing MySQL-backed API.

- The catalogue filters, paginates and prints correctly; PHP generates a valid CSV report.

- Source code, SQL schema, proposal/report PDFs and repository links are available for evaluation.

09 Application of web development concepts

Course concept	Practical application
HTML semantics and forms	Structured pages, labelled filters, admin forms and tabular catalogue data.
CSS and responsive design	Flexbox/Grid, Bootstrap breakpoints, media queries and horizontal table scrolling.
JavaScript and events	Reader controls, form handling, filter events and React state updates.
AJAX / JSON / REST	Fetch-based communication with Express endpoints such as GET /api/pdfs.
Backend development	Express routes/controllers plus a separate PHP command-line reporting workflow.
Database connectivity / CRUD	MySQL connection pooling, parameterized operations and schema files.
Authentication and access	Google token verification, signed sessions, cookies, CSRF and role middleware.
Testing and debugging	Syntax checks, browser scenarios, CSV tests, API errors and retry states.
Version control / deployment	GitHub CI, Cloudflare Pages, hosted Express API, SEO metadata and caching.

10 / WEB DEVELOPMENT

References

1. Project source code and SQL schema
https://github.com/padam421/E-LIBRARY

2. Project setup and architecture notes
https://github.com/padam421/E-LIBRARY/tree/main/docs

3. MDN Web Docs - HTML, CSS and JavaScript
https://developer.mozilla.org/en-US/docs/Web

4. React - createElement and browser rendering
https://react.dev/reference/react/createElement

5. Bootstrap 5.3 - getting started
https://getbootstrap.com/docs/5.3/getting-started/introduction/

6. Express - API reference
https://expressjs.com/en/4x/api.html

7. PHP - CSV output with fputcsv
https://www.php.net/manual/en/function.fputcsv.php

8. MySQL 8.0 Reference Manual
https://dev.mysql.com/doc/refman/8.0/en/

Academic requirements: the user-supplied Minor Project Guidelines and Google Form screenshots for the Web Development course. No public guideline URL was provided.

Feasibility and deployment

The existing static frontend targets Cloudflare Pages and the primary Express API uses a separate host. PHP reporting runs locally or on a PHP-capable host. External Google, AI, media and Razorpay integrations depend on their configured credentials and services.

Evaluation demonstration

Demonstrate discovery, catalogue filters, one available reader, administrator book management and PHP CSV export. Explain the client-server request flow, database relationships, validation and role checks. The accompanying final report records implementation evidence and the validation completed for this update.